

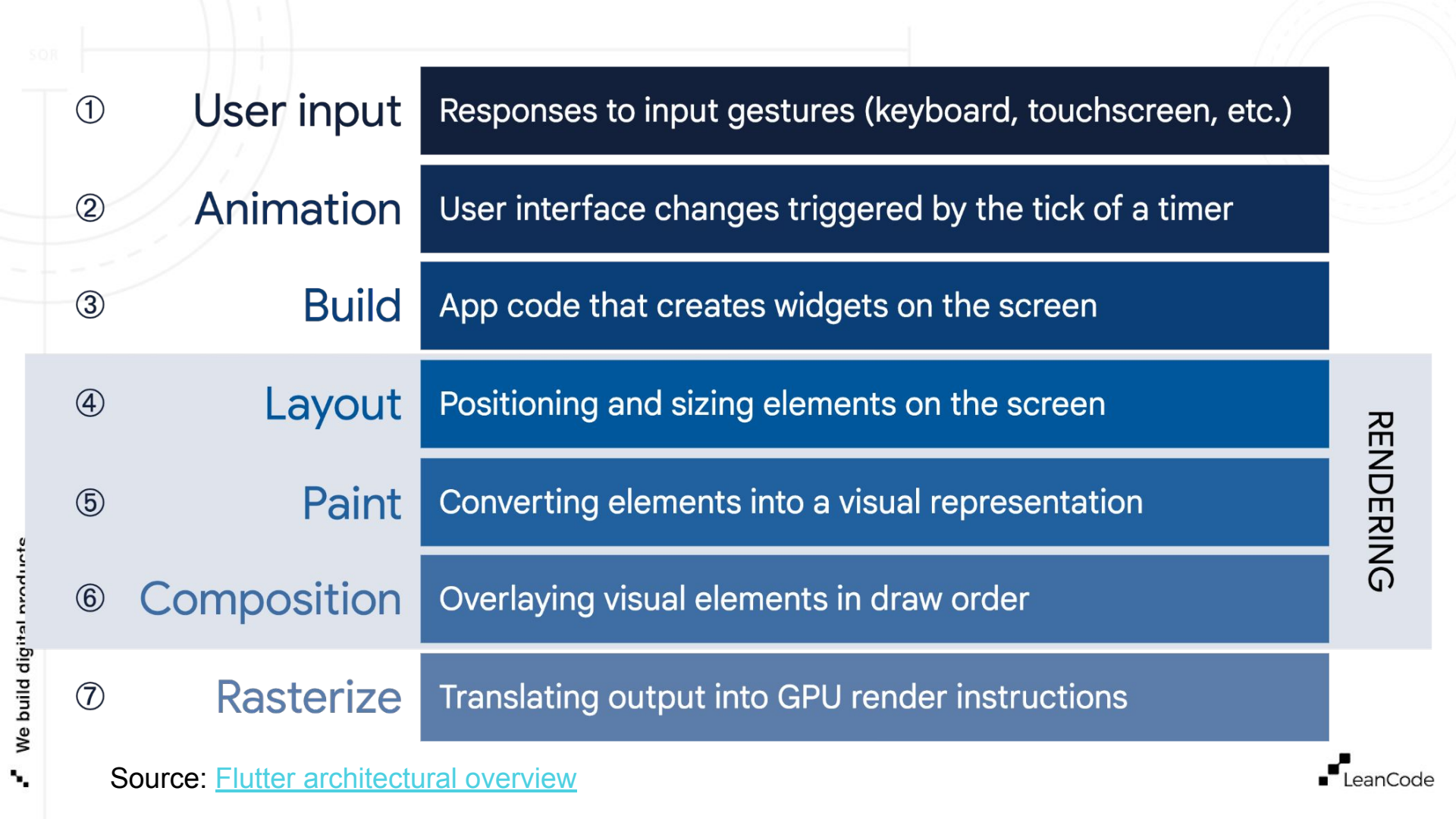
Giving context to BuildContext



Tomek Koter
senior dev @ LeanCode
tomasz.koter@leancode.pl

Plan na dziś

- co kryje się pod spodem widgetów
- interakcje
- zarządzanie stanem lokalnym
- przekazywanie danych przez kontekst - InheritedWidget
- reagowanie na zmiany

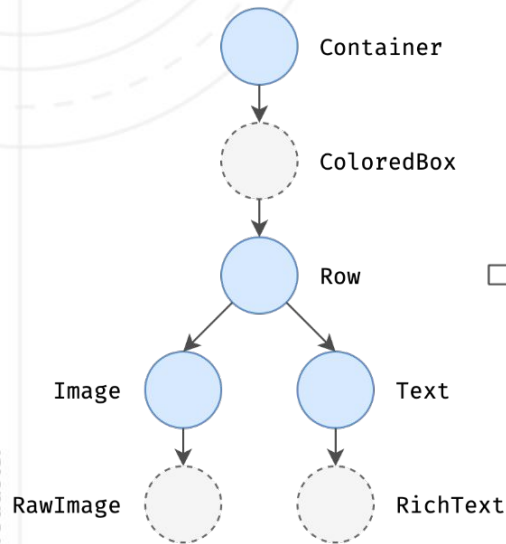
- 
- The diagram illustrates the Flutter rendering pipeline as a vertical sequence of seven steps. Steps 1-3 are on a white background, while steps 4-7 are grouped under a light blue 'RENDERING' label on the right. The steps are: 1. User input (dark blue bar), 2. Animation (dark blue bar), 3. Build (dark blue bar), 4. Layout (medium blue bar), 5. Paint (medium blue bar), 6. Composition (medium blue bar), and 7. Rasterize (light blue bar).
- ① **User input** Responses to input gestures (keyboard, touchscreen, etc.)
 - ② **Animation** User interface changes triggered by the tick of a timer
 - ③ **Build** App code that creates widgets on the screen
 - ④ **Layout** Positioning and sizing elements on the screen
 - ⑤ **Paint** Converting elements into a visual representation
 - ⑥ **Composition** Overlaying visual elements in draw order
 - ⑦ **Rasterize** Translating output into GPU render instructions

RENDERING

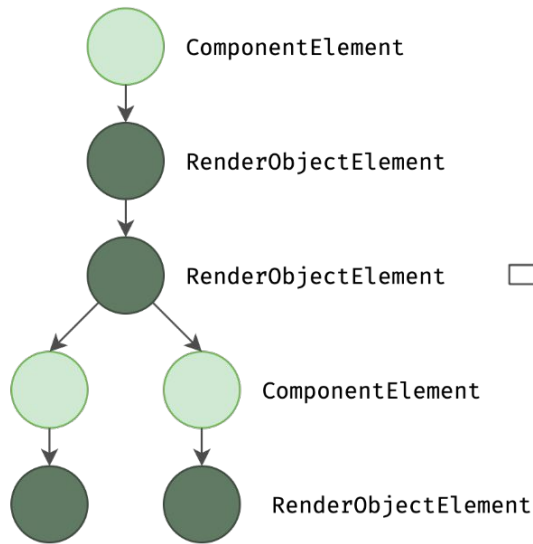


```
class MyWidget extends StatelessWidget {  
  const MyWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.amber.shade100,  
      width: 100,  
      height: 100,  
      child: Row(  
        children: [  
          Image.network('https://picsum.photos/200/300'),  
          Text('Hello World!'),  
        ],  
      ),  
    );  
  }  
}
```

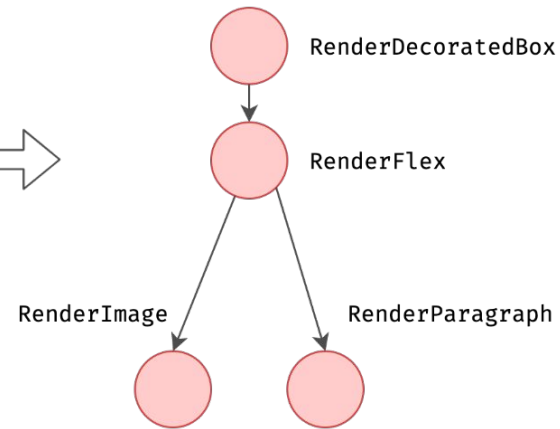
Widgets



Element Tree



Render Tree



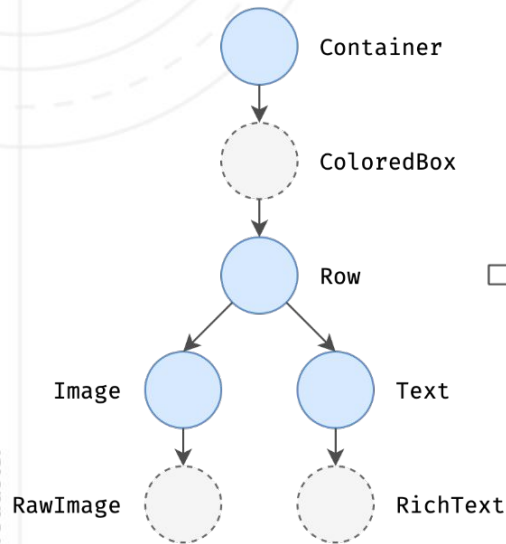
```

246 class Container extends StatelessWidget {
381   @override
382   Widget build(BuildContext context) {
383     Widget? current = child;
384
385     if (child == null && (constraints == null || !constraints!.isTight)) {
386       current = LimitedBox(
387         maxWidth: 0.0,
388         maxHeight: 0.0,
389         child: ConstrainedBox(constraints: const BoxConstraints.expand()),
390       );
391     } else if (alignment != null) {
392       current = Align(alignment: alignment!, child: current);
393     }
394
395     final EdgeInsetsGeometry? effectivePadding = _paddingIncludingDecoration;
396     if (effectivePadding != null) {
397       current = Padding(padding: effectivePadding, child: current);
398     }
399
400     if (color != null) {
401       current = ColoredBox(color: color!, child: current);
402     }
403

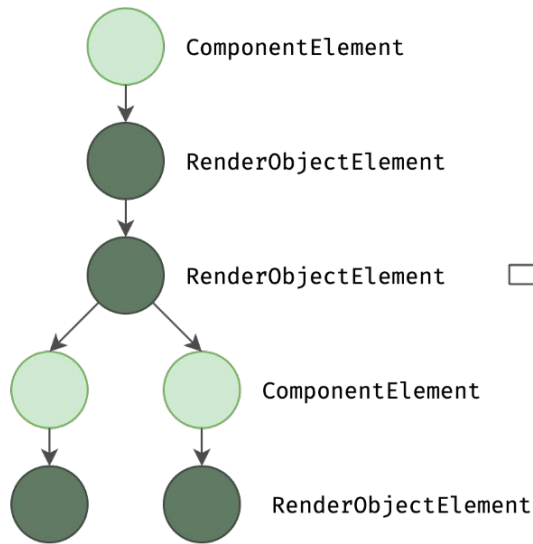
```

```
246 class Container extends StatelessWidget {  
382   Widget build(BuildContext context) {  
  
418     }  
419  
420     if (foregroundDecoration != null) {  
421       current = DecoratedBox(  
422         decoration: foregroundDecoration!,  
423         position: DecorationPosition.foreground,  
424         child: current,  
425       );  
426     }  
427  
428     if (constraints != null) {  
429       current = ConstrainedBox(constraints: constraints!, child: current);  
430     }  
431  
432     if (margin != null) {  
433       current = Padding(padding: margin!, child: current);  
434     }  
435  
436     if (transform != null) {  
437       current = Transform(transform: transform!, alignment: transformAlignment, child: current);  
438     }  
439  
440     return current!;  
441   }  
442 }
```

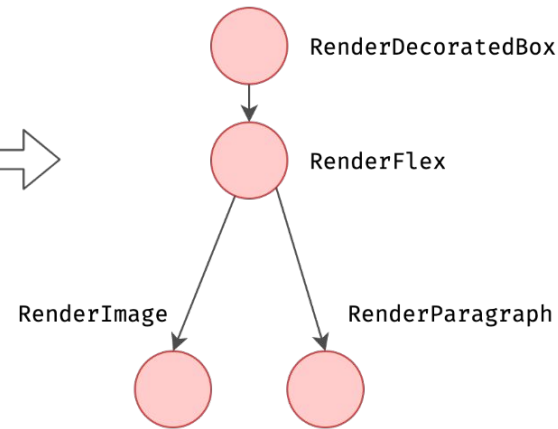
Widgets



Element Tree



Render Tree



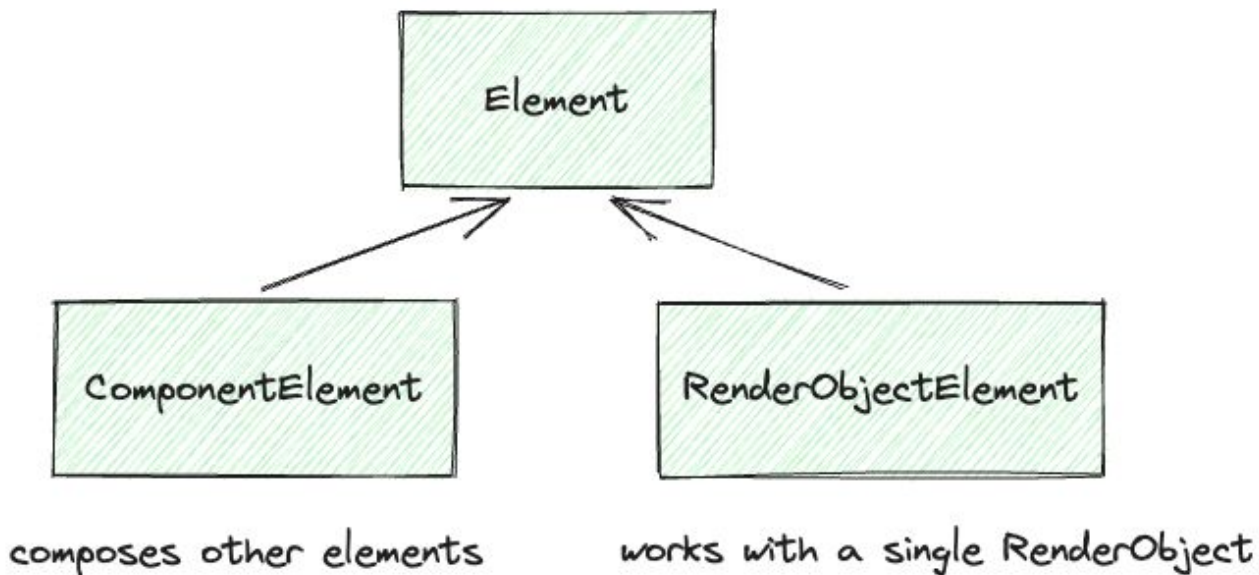
Widgets

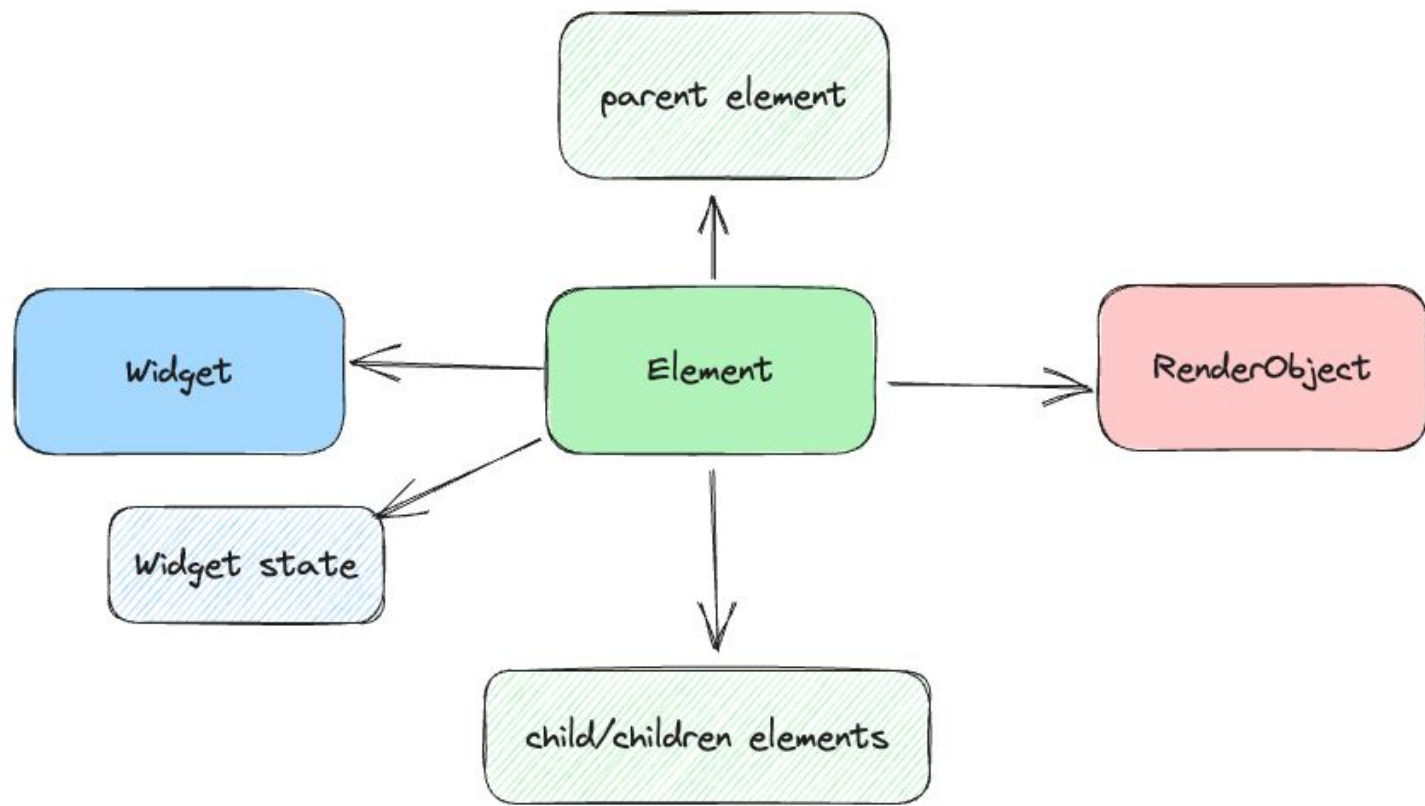
- are **immutable** – never change; always describe the same thing
- are **configuration** for elements
- are **recipes** for subtrees, parts of application
- are not associated with any specific part of the tree
- are (almost) **pure**, plain data objects
- think cheap, lightweight, disposable, ephemeral, impermanent
- one instance can be reused in **multiple** places; one instance can configure **multiple** elements

Elements

“ An instantiation of a Widget at a particular location in the tree.

[Element class - widgets library - Dart API](#)





Elements

- are associated with a specific location in the tree
- persistent across rebuilds
- have a lifecycle
- can access ancestors and descendants
- can be reconfigured with a different widget
- keep track of their widgets and their state (if widget is stateful)

RenderObjects

Responsibilities:

- layout
- painting
- semantics (accessibility)
- operate in terms of a canvas

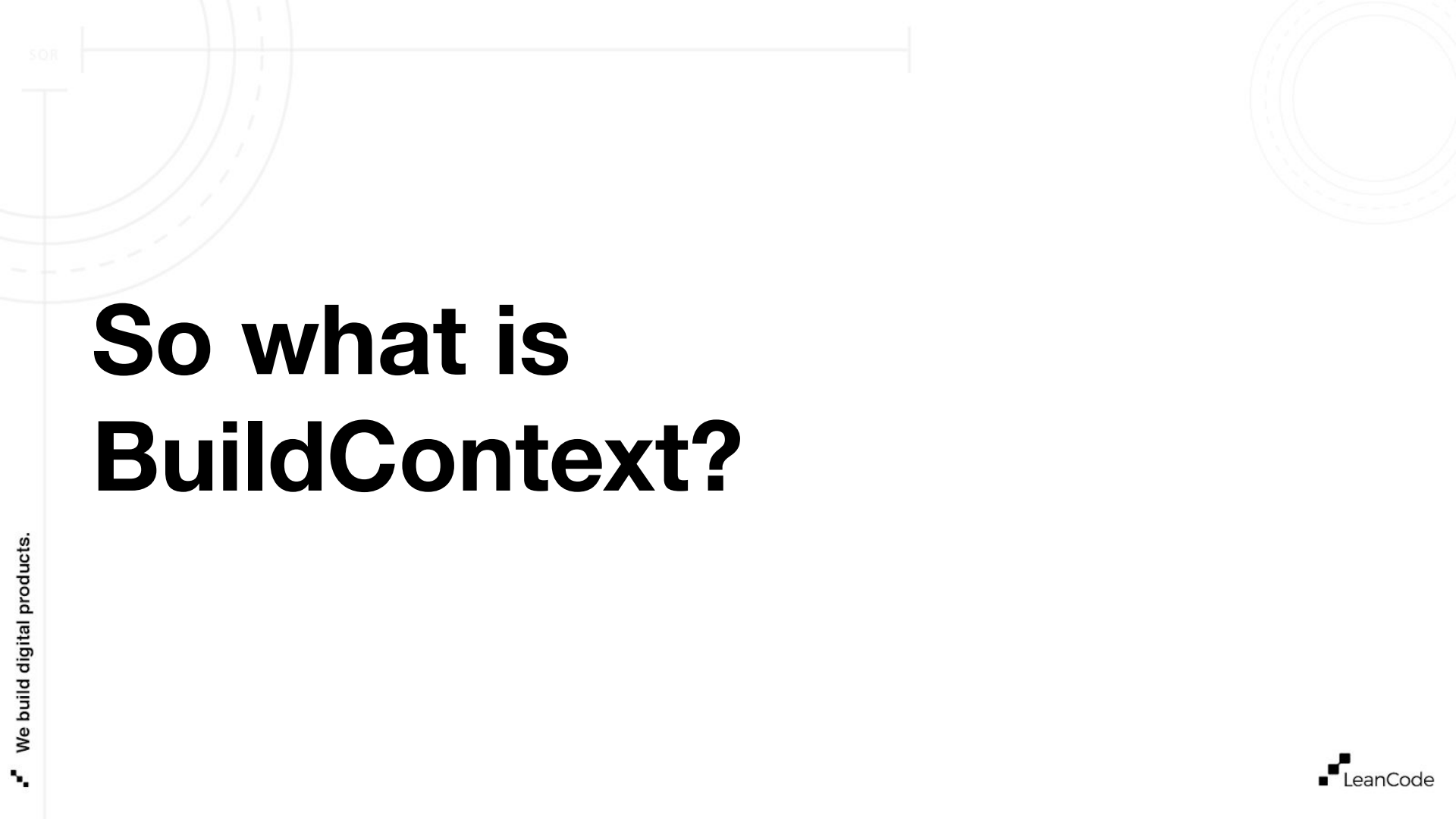
RenderObjects



```
/// Compute the layout for this render object.  
/// [...]  
@protected  
void performLayout();  
  
/// Paint this render object into the given context at the given offset.  
/// [...]  
void paint(PaintingContext context, Offset offset);
```

RenderObjects

- ! Low(er) level
- ! Rarely touched
- ! Prefer builtin widgets unless you need custom layouts



So what is BuildContext?

Element class

Inheritance

[Object](#) > [DiagnosticableTree](#) > [Element](#)

Implemented types

[BuildContext](#)

[Element class - widgets library - Dart API](#)

[BuildContext class - widgets library - Dart API](#)

[BuildContext](#) objects are actually [Element](#) objects. The [BuildContext](#) interface is used to discourage direct manipulation of [Element](#) objects.

Implementers

[Element](#)

① User input Responses to input gestures (keyboard, touchscreen, etc.)

② Animation User interface changes triggered by the tick of a timer

③ Build App code that creates widgets on the screen

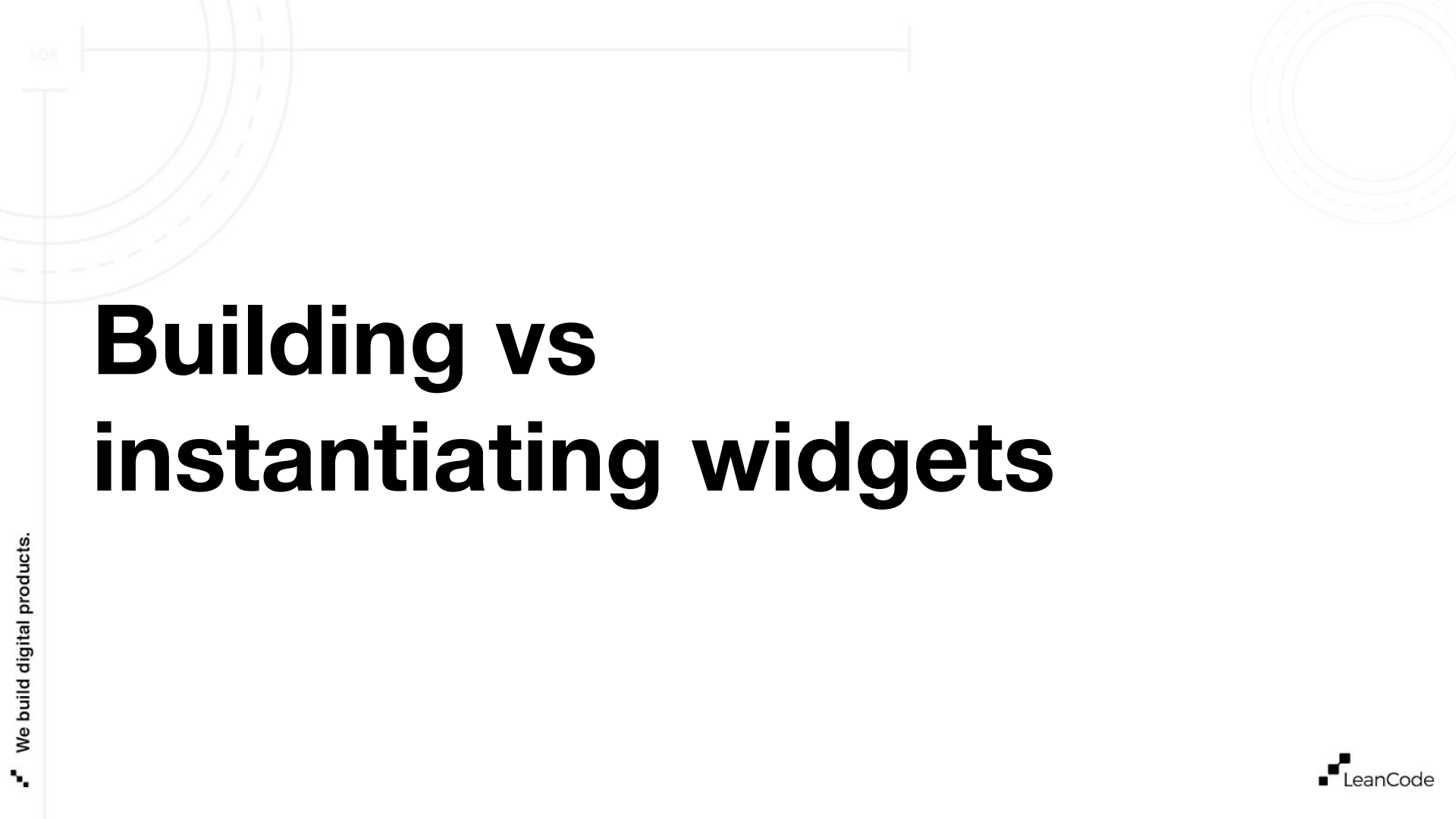
④ Layout Positioning and sizing elements on the screen

⑤ Paint Converting elements into a visual representation

⑥ Composition Overlaying visual elements in draw order

⑦ Rasterize Translating output into GPU render instructions

RENDERING



Building vs instantiating widgets

Constructor just instantiates the widget object



```
final widget = MyWidget();
```

Constructor just instantiates the widget object

```
class MyWidget extends StatelessWidget {  
  const MyWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.amber.shade100,  
      width: 100,  
      height: 100,  
      child: Row(  
        children: [  
          Image.network('https://picsum.photos/200/300'),  
          Text('Hello World!'),  
        ],  
      ),  
    );  
  }  
}
```

Constructor just instantiates the widget object

```
// ...  
///      several times over their lifetime.  
/// * [InheritedWidget], for widgets that introduce ambient state that can  
///      be read by descendant widgets.  
abstract class StatelessWidget extends Widget {  
  /// Initializes [key] for subclasses.  
  const StatelessWidget({super.key});  
  
  /// Creates a [StatelessElement] to manage this widget's location in the tree.  
  // ...
```

Constructor just instantiates the widget object

```
@immutable  
abstract class Widget extends DiagnosticableTree {  
  /// Initializes [key] for subclasses.  
  const Widget({this.key});  
}
```

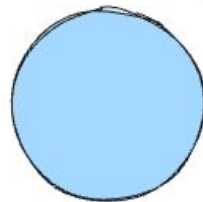
Constructor just instantiates the widget object

```
abstract class DiagnosticableTree with Diagnosticable {  
  /// Abstract const constructor. This constructor enables subclasses to provide  
  /// const constructors so that they can be used in const expressions.  
  const DiagnosticableTree();  
}
```


- ! build method - not called**
- ! Flutter - not involved at all**
- ! Just a lightweight object is created**

build method

`runApp(MyWidget())`



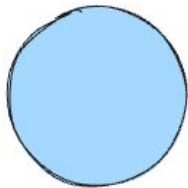
MyWidget

effects:

- a) MyWidget widget instantiated
- b) framework begins init
- c) MyWidget registered with framework as root
- d) MyWidget needs to be built

build method

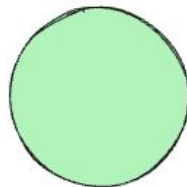
MyWidget



`MyWidget.createElement()`



ComponentElement



build method

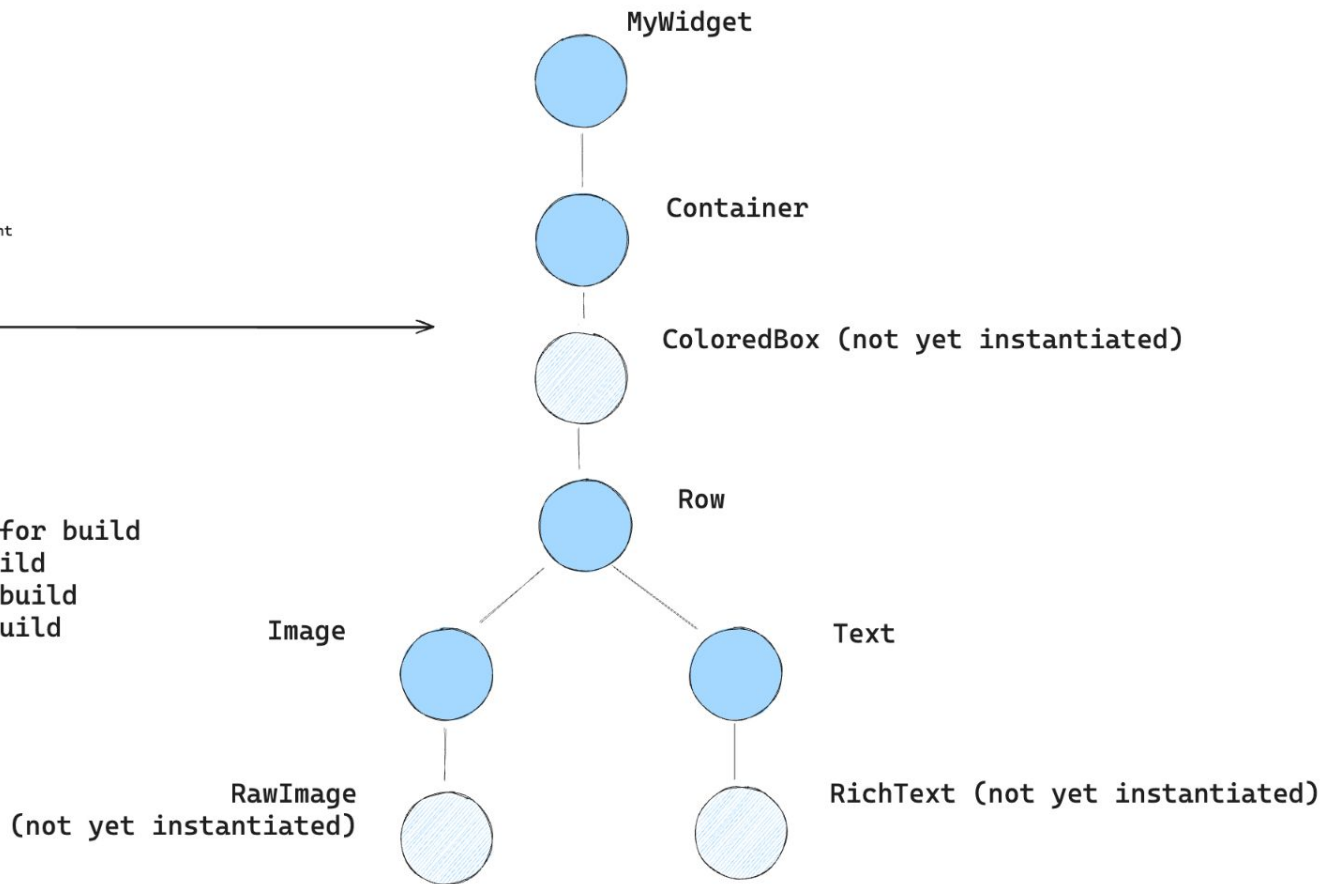
context =  ComponentElement

MyWidget.build(context)



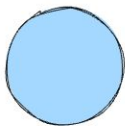
effects, as per build method:

- a) Container instantiated and marked for build
- b) Row instantiated and marked for build
- c) Image instantiated and marked for build
- d) Text instantiated and marked for build



build method

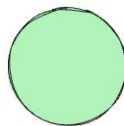
Container



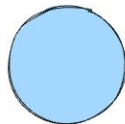
`Container.createElement()`



ComponentElement



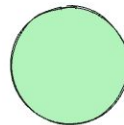
Row



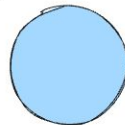
`Row.createElement()`



RenderObjectElement



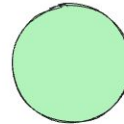
Image



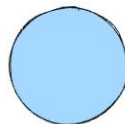
`Image.createElement()`



ComponentElement



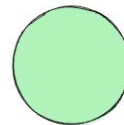
Text



`Text.createElement()`



ComponentElement



build method

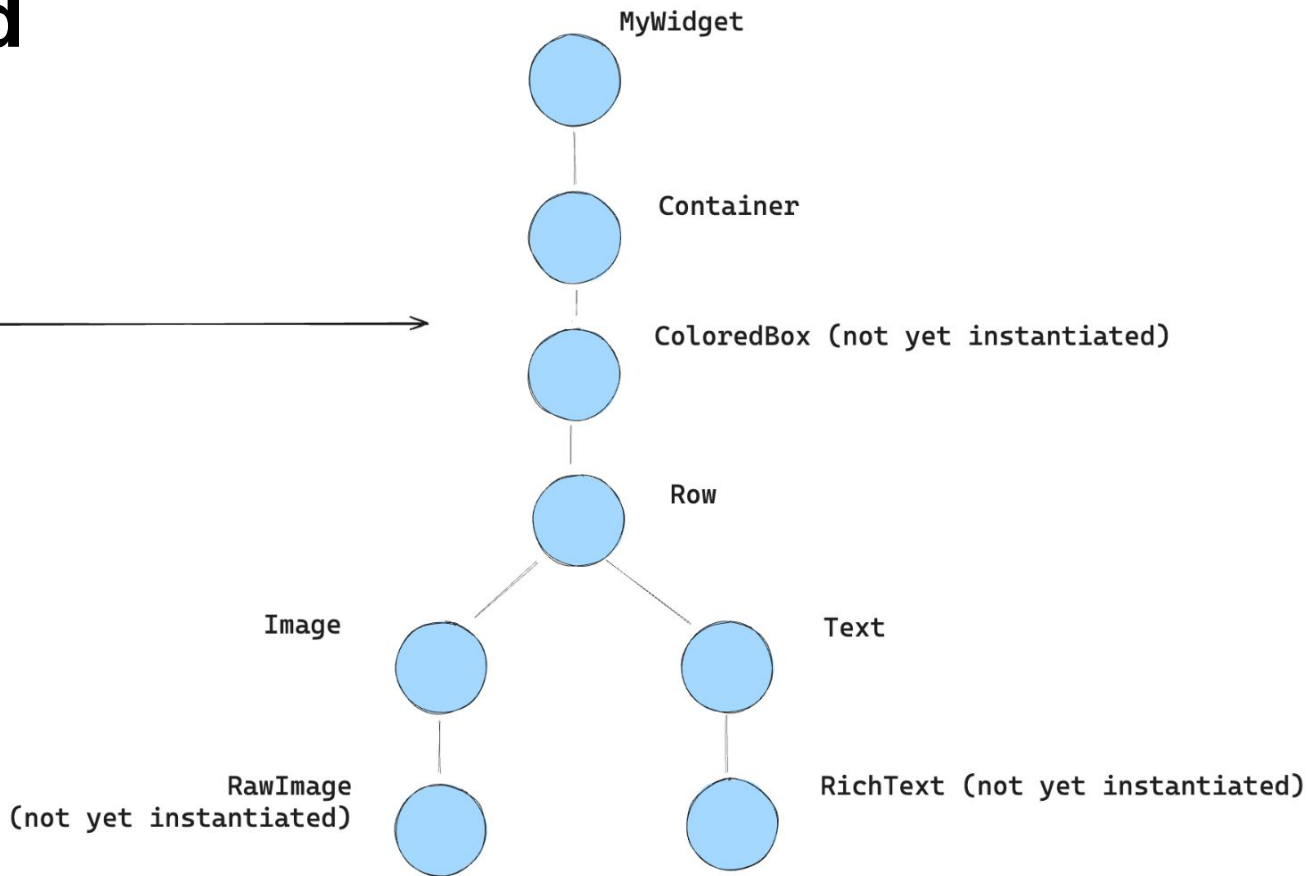
context =  Element

Container.build(context)

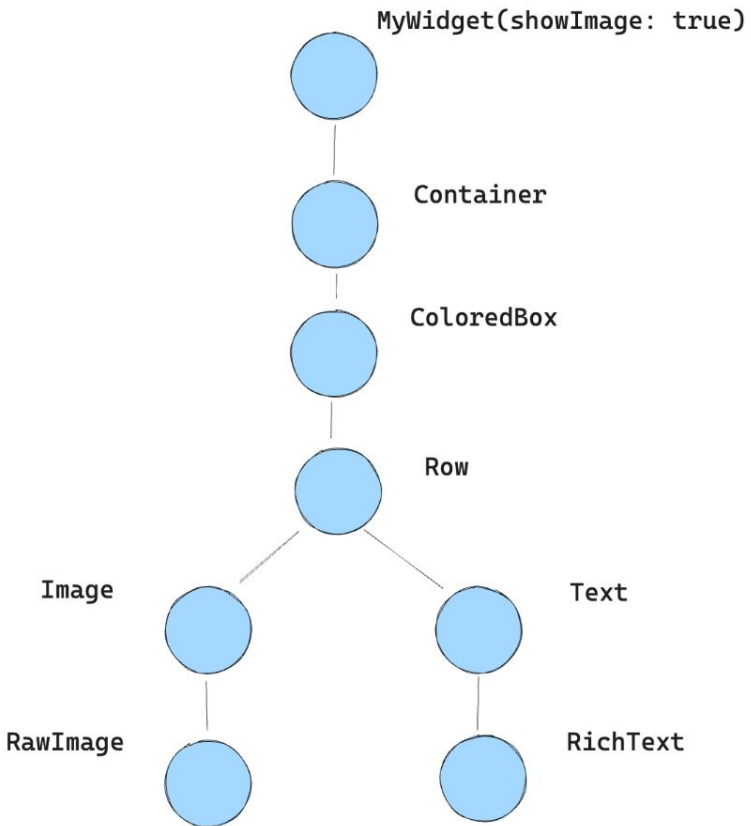
Row.build(context)

Image.build(context)

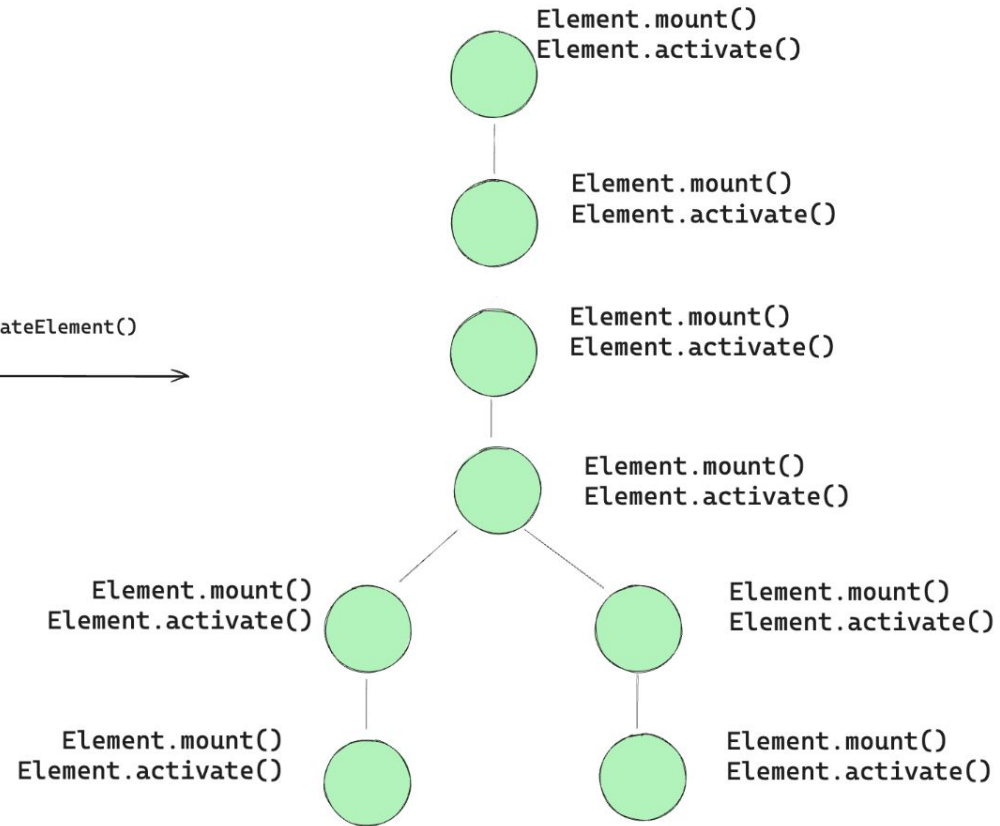
Text.build(context)

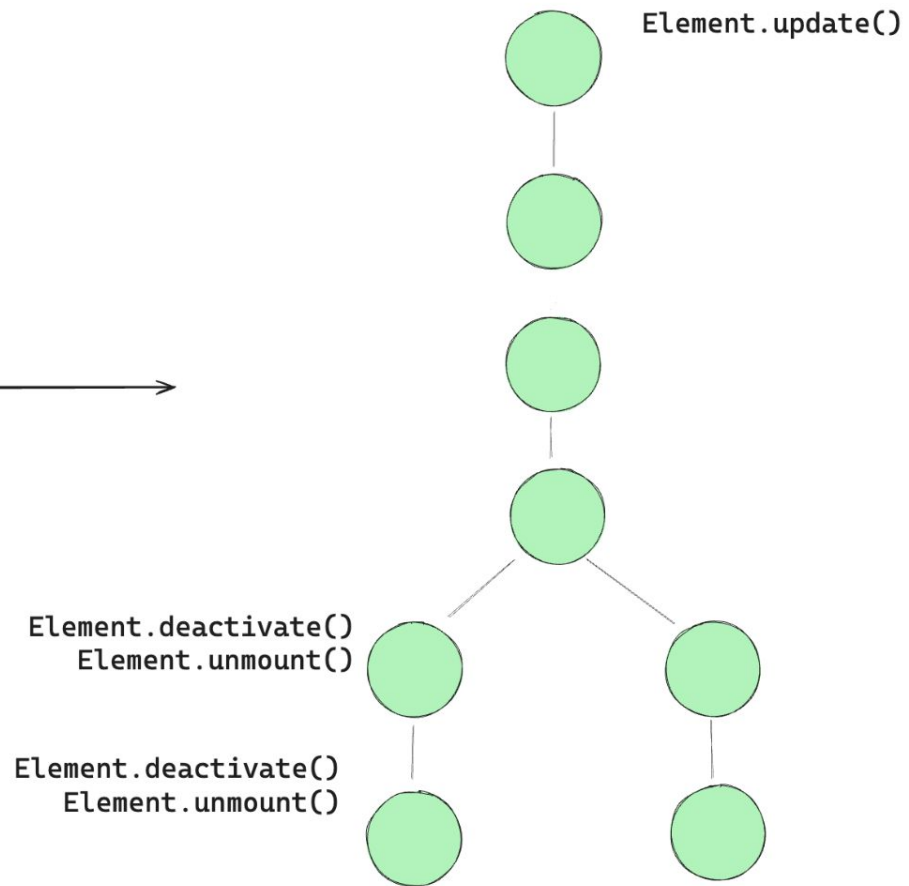
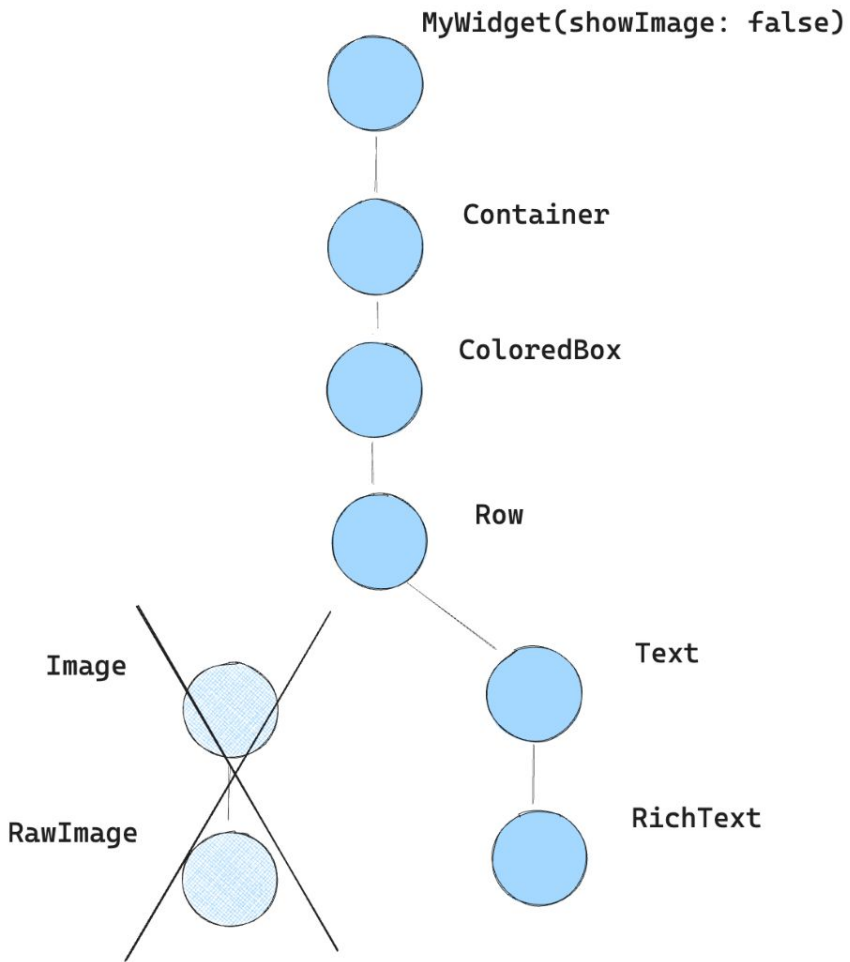


```
class MyWidget2 extends StatelessWidget {  
  const MyWidget2({super.key, required this.showImage});  
  
  final bool showImage;  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.amber.shade100,  
      width: 100,  
      height: 100,  
      child: Row(  
        children: [  
          if (showImage) Image.network('https://picsum.photos/200/300'),  
          Text('Hello World!'),  
        ],  
      ),  
    );  
  }  
}
```

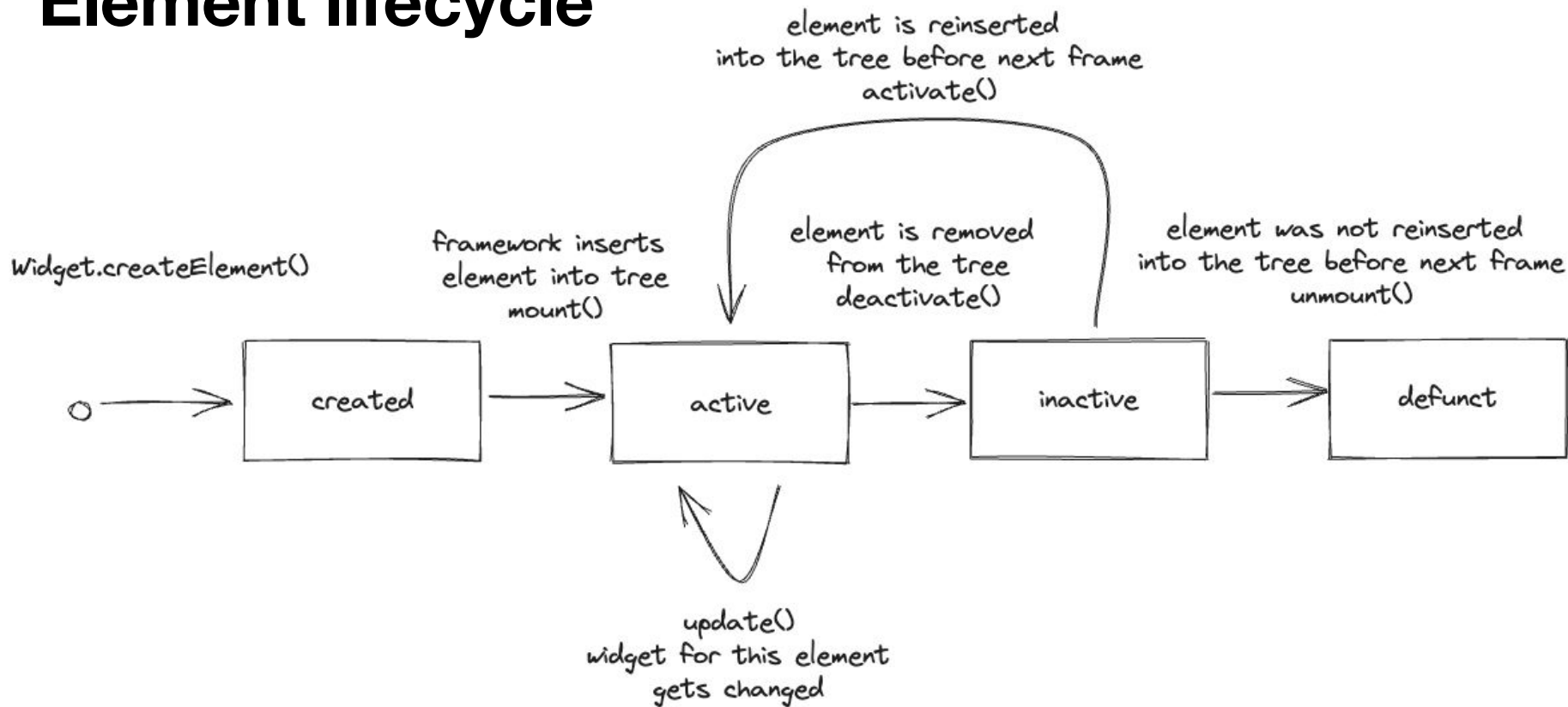


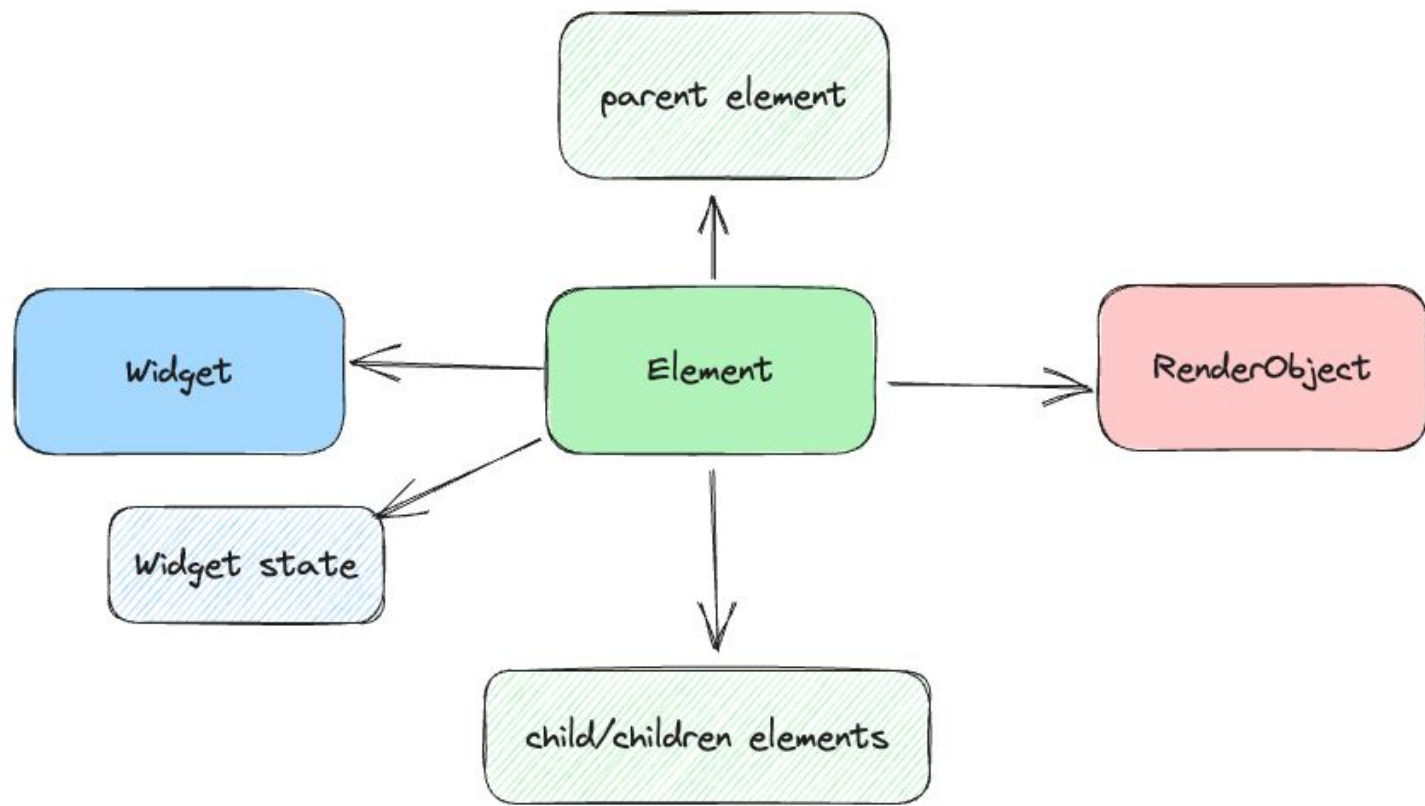
`Widget.createElement()`



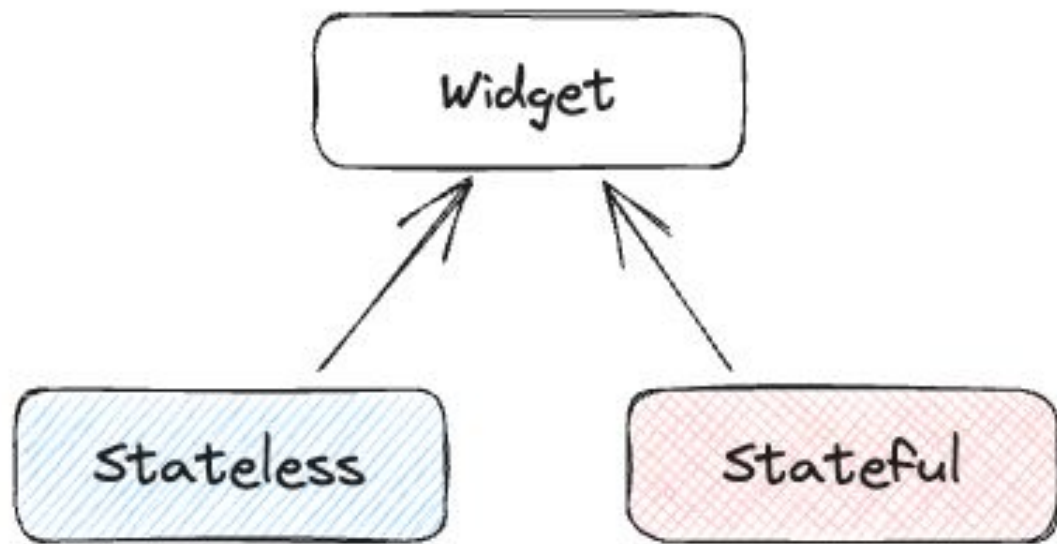


Element lifecycle





Enter StatefulWidget



Stateless widget



StatelessWidget

immutable properties

build method

Stateful widget



StatefulWidget

immutable properties

createState method



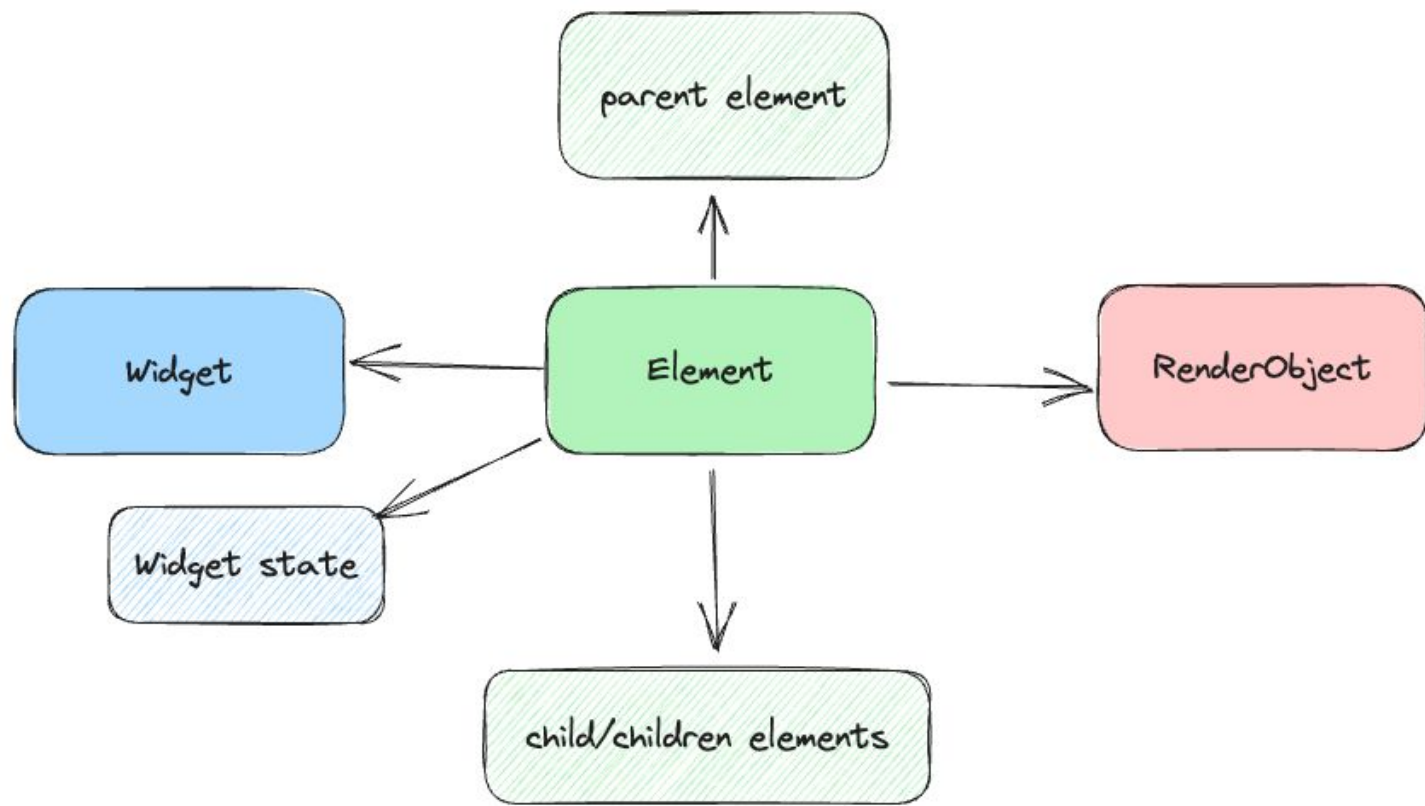
State

mutable properties

'widget' property

build method





Making a widget stateful

```
class StatelessCounter extends StatelessWidget {  
  const StatelessCounter({  
    super.key,  
    required this.value,  
    required this.onChanged,  
  });  
  
  final int value;  
  final ValueChanged<int> onChanged;  
  
  @override  
  Widget build(BuildContext context) {  
    return Row(  
      children: [  
        Text(value.toString()),  
        ElevatedButton(onPressed: () => onChanged(value + 1), child: Text('+')),  
        ElevatedButton(onPressed: () => onChanged(value - 1), child: Text('-')),  
      ],  
    );  
  }  
}
```




```
class StatefulCounter extends StatefulWidget {
  const StatefulCounter({
    super.key,
    required this.initialValue,
  });

  final int initialValue;

  @override
  State<StatefulCounter> createState() =>
    _StatefulCounterState();
}
```



```
class _StatefulCounterState extends State<StatefulCounter> {
  late int _value;

  @override
  void initState() {
    super.initState();
    _value = widget.initialValue;
  }

  void _increment() {
    setState(() => _value++);
  }

  void _decrement() {
    setState(() => _value--);
  }

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text(_value.toString()),
        ElevatedButton(onPressed: _increment, child: Text('+')),
        ElevatedButton(onPressed: _decrement, child: Text('-')),
      ],
    );
  }
}
```

Or simply

```
class StatelessWidget {  
    const Stateless  
    super.key,  
    required this  
    required this  
});  
  
final int value;  
final ValueChanged<int> onChanged;
```

Move



Move 'StatelessCounter' to file

More Actions...

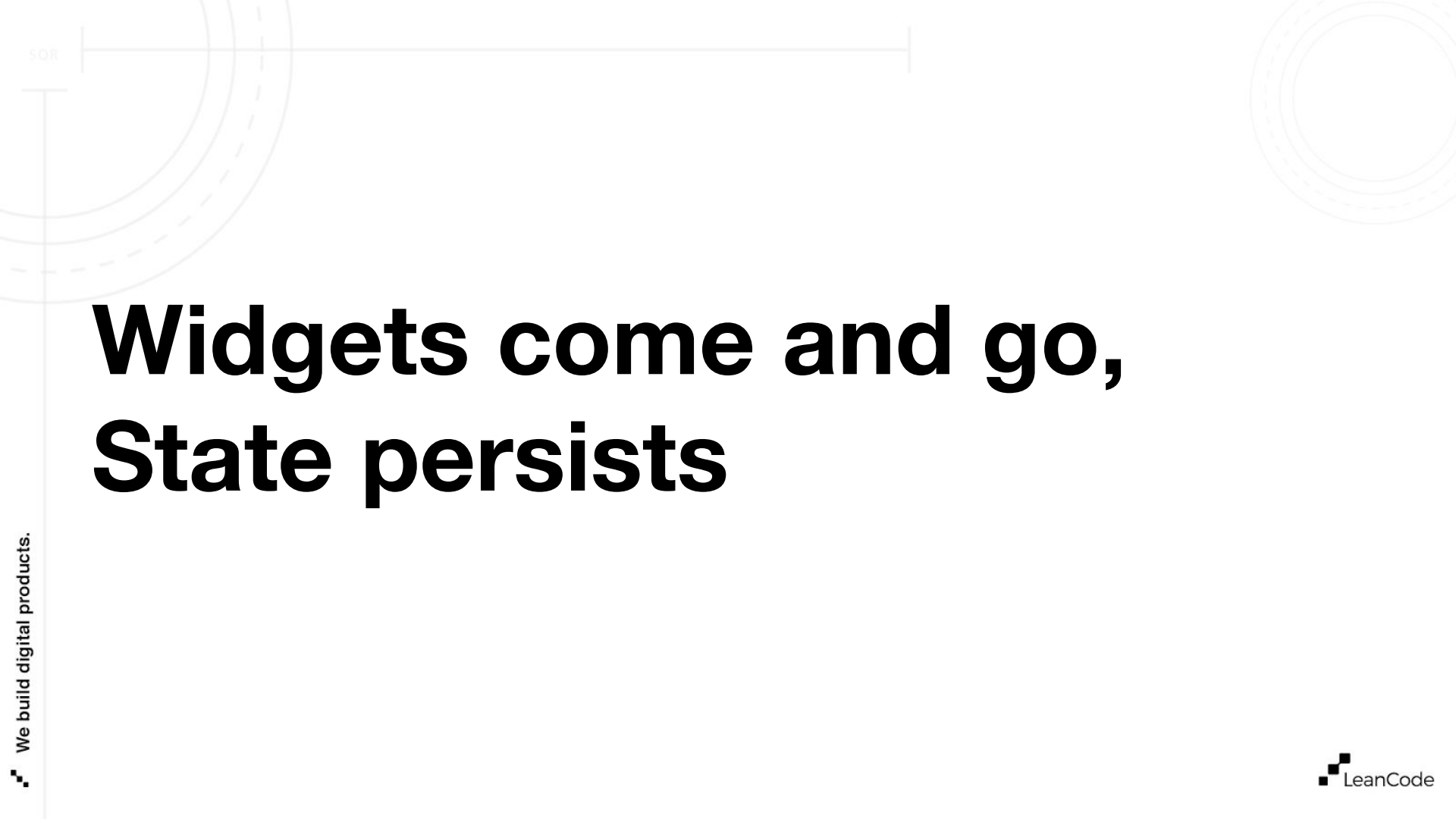


Convert to StatefulWidget

Making a widget stateful

Note the differences from StatelessWidget:

- stateful widgets consist of **two** classes: StatefulWidget and State
- the **build** method is in State, not in StatefulWidget
- properties on StatefulWidget are **immutable** – you can't change them
- the actual **state** (mutable properties) resides in State



Widgets come and go, State persists

Changing state

use the setState method

```
class CounterState extends State<Counter> {  
  int _value = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return TextButton(  
      style: TextButton.styleFrom(  
        textStyle: TextStyle(color: widget.color),  
      ),  
      onPressed: () {  
        setState(() {  
          _value++;  
        });  
      },  
      child: Text('Counter: $_value'),  
    );  
  }  
}
```

State's lifecycle

State.build() can be called in this state

StatefulWidget.createState()
+
constructor State()

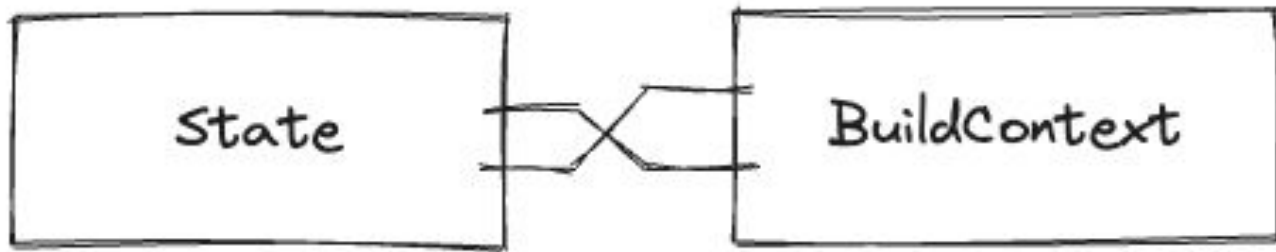
initState()

when not activated
before end of frame
dispose()

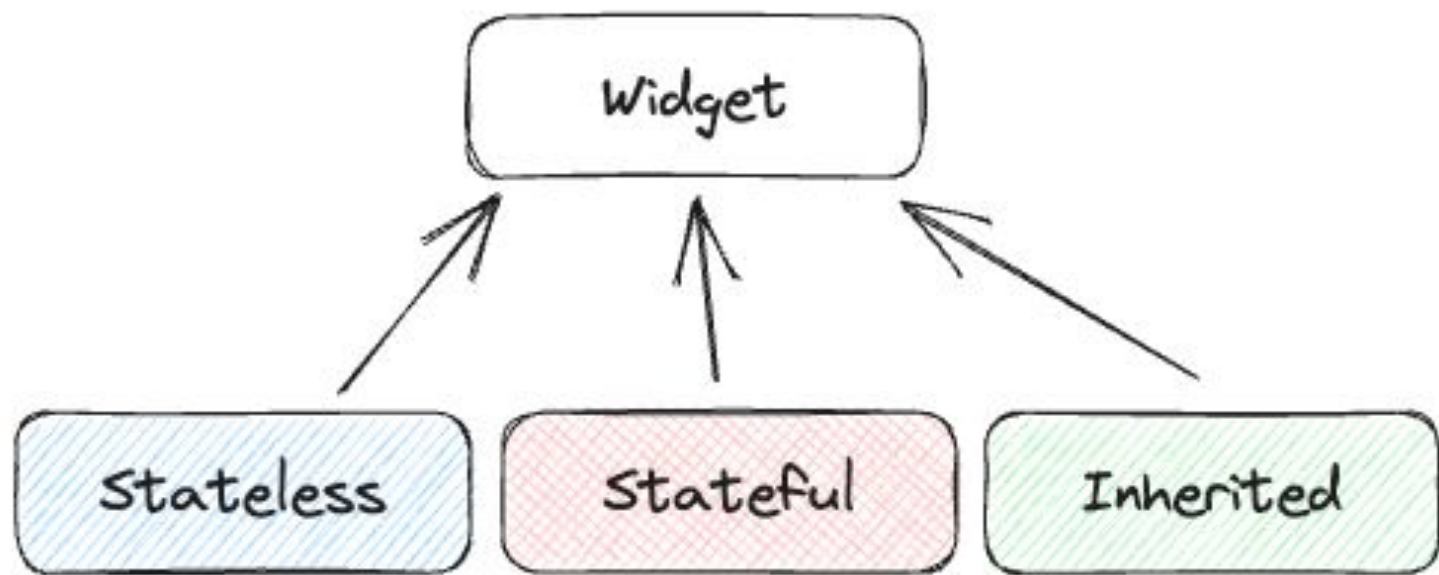


didChangeDependencies()

State is bound to a single BuildContext (Element)



InheritedWidget



Why it exists



```
Widget build(BuildContext context) {  
  return AppRoot(  
    userInfo: userInfo,  
    child: HomePage(  
      userInfo: userInfo,  
      child: ProfileCard(userInfo: userInfo),  
    ),  
  );  
}
```

Why it exists



```
class NavigatorExample extends StatelessWidget {  
  const NavigatorExample({super.key, required this.userInfo});  
  
  final UserInfo userInfo;  
  
  @override  
  Widget build(BuildContext context) {  
    return Navigator(  
      /* ??? How do we pass the userInfo to the next screen? */  
    );  
  }  
}
```



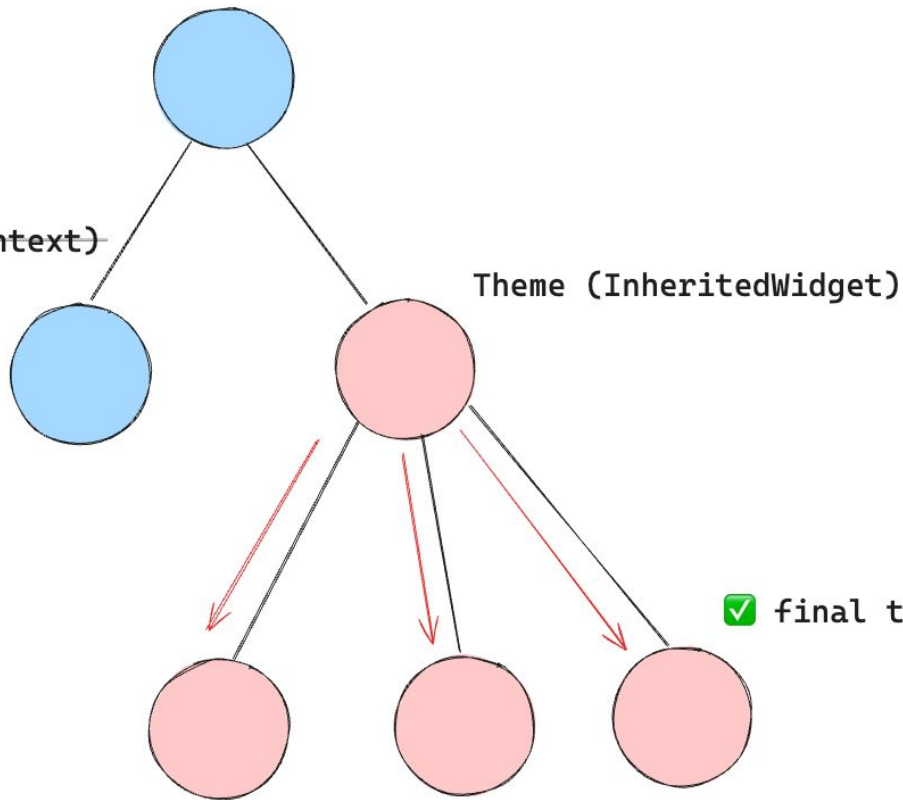
```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    final theme = ThemeData.light();  
  
    return MaterialApp(  
      theme: theme,  
      home: Scaffold(  
        backgroundColor: theme.scaffoldBackgroundColor,  
        body: Center(child: Text('Hello World!')),  
        floatingActionButton: FloatingActionButton(  
          onPressed: () {},  
          foregroundColor: theme.floatingActionButtonTheme.foregroundColor,  
          backgroundColor: theme.floatingActionButtonTheme.backgroundColor,  
        ),  
      ),  
    );  
  }  
}
```



Data flows down

~~final theme = Theme.of(context)~~

✗ inaccessible



✓ final theme = Theme.of(context)

InheritedWidget

```
class AppThemeProvider extends InheritedWidget {  
  const AppThemeProvider({  
    super.key,  
    required this.appTheme,  
    required super.child,  
  });  
  
  final AppTheme appTheme;  
  
  static AppTheme? of(BuildContext context) {  
    return context  
      .dependOnInheritedWidgetOfExactType<AppThemeProvider>()  
      ?.appTheme;  
  }  
  
  @override  
  bool updateShouldNotify(AppThemeProvider oldWidget) {  
    return oldWidget.appTheme != appTheme;  
  }  
}
```

InheritedWidget

1 Extend InheritedWidget

2 Add properties to pass to descendants

4 Limit update frequency for better performance

3 Add idiomatic `of` method

```
class AppThemeProvider extends InheritedWidget {  
  const AppThemeProvider({  
    super.key,  
    required this.appTheme,  
    required super.child,  
  });  
  
  final AppTheme appTheme;  
  
  static AppTheme? of(BuildContext context) {  
    return context  
      .dependOnInheritedWidgetOfExactType<AppThemeProvider>()  
      ?.appTheme;  
  }  
  
  @override  
  bool updateShouldNotify(AppThemeProvider oldWidget) {  
    return oldWidget.appTheme != appTheme;  
  }  
}
```

InheritedWidget



```
@override
Widget build(BuildContext context) {
  return AppThemeProvider(
    appTheme: theme,
    child: const SomeComplexHierarchy(),
  );
}
```


InheritedWidget

```
class AppText extends StatelessWidget {  
  const AppText(this.text, {super.key});  
  
  final String text;  
  
  @override  
  Widget build(BuildContext context) {  
    final theme = AppThemeProvider.of(context)!;  
  
    return Text(  
      text,  
      style: TextStyle(color: theme.textColor),  
    );  
  }  
}
```

Provider [pub.dev]

InheritedWidget but easier

Provider

No need to define custom InheritedWidget

→ use ready-made, generic Provider

```
@override
Widget build(BuildContext context) {
  return Provider.value(
    value: theme,
    child: const SomeComplexHierarchy(),
  );
}
```

Provider

```
class AppText extends StatelessWidget {  
  const AppText(this.text, {super.key});  
  
  final String text;  
  
  @override  
  Widget build(BuildContext context) {  
    final theme = context.watch<AppTheme>();  
    // same as this:  
    // final theme = Provider.of<AppTheme>(context);  
  
    return Text(  
      text,  
      style: TextStyle(color: theme.textColor),  
    );  
  }  
}
```

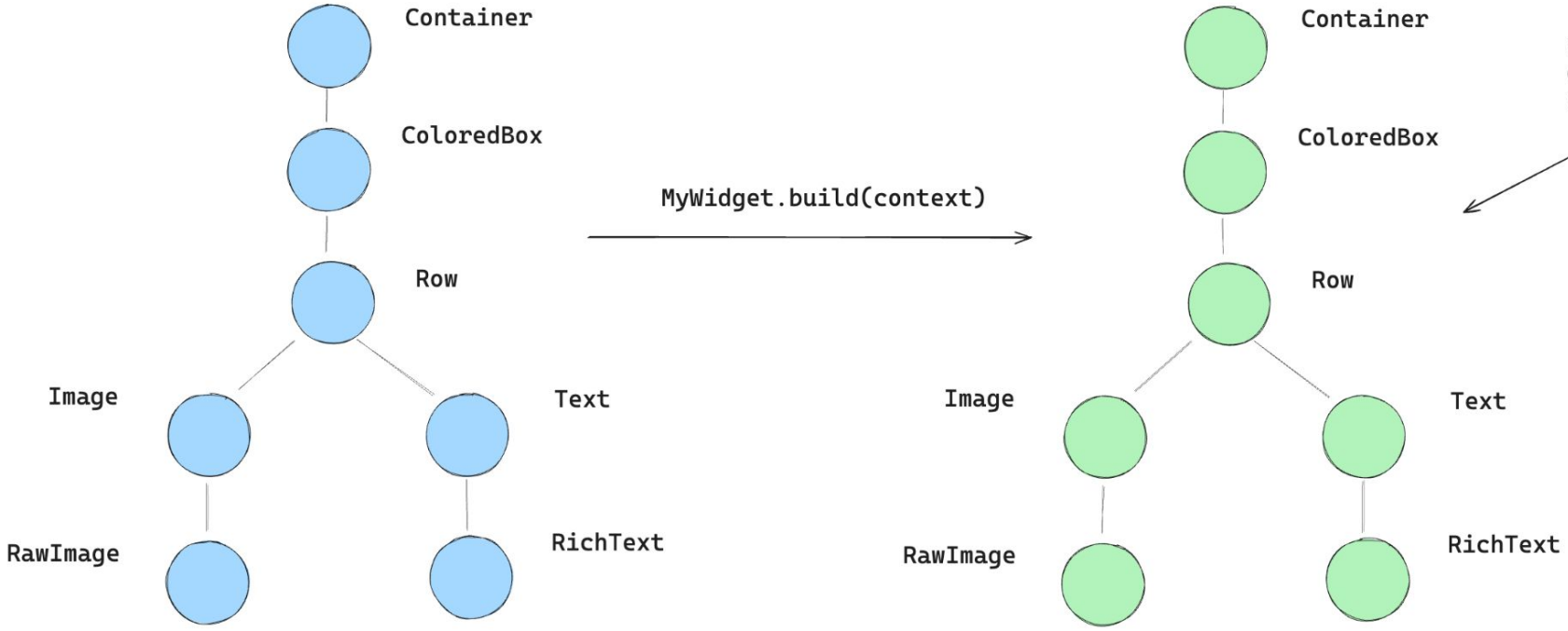
Keys

Useful info here, including 10-min youtube video
[Key class - foundation library - Dart API \(flutter.dev\)](#)

Keys are widget identifiers



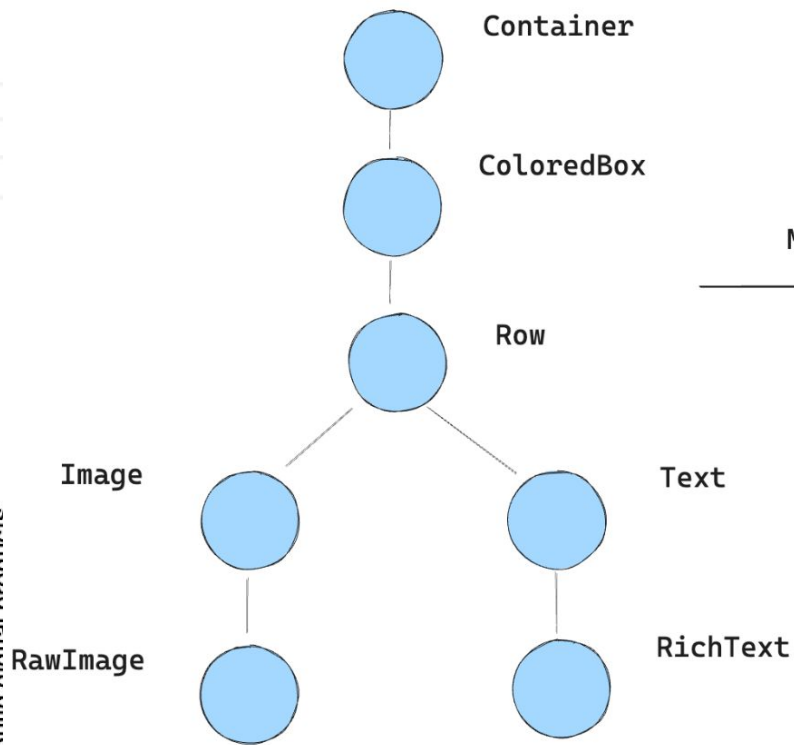
```
class MyWidget extends StatelessWidget {  
  const MyWidget({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.amber.shade100,  
      width: 100,  
      height: 100,  
      child: Row(  
        children: [  
          Image.network('https://picsum.photos/200/300'),  
          Text('Hello World!'),  
        ],  
      ),  
    );  
  }  
}
```



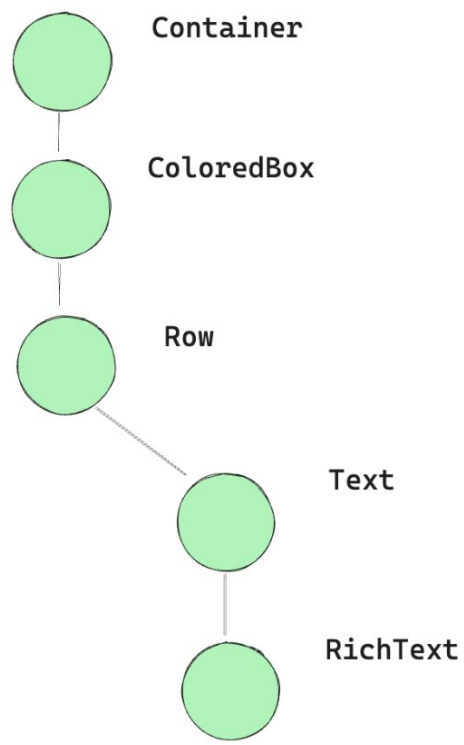
same widgets,
different
instances


```
class MyWidget2 extends StatelessWidget {  
  const MyWidget2({super.key, required this.showImage});  
  
  final bool showImage;  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.amber.shade100,  
      width: 100,  
      height: 100,  
      child: Row(  
        children: [  
          if (showImage) Image.network('https://picsum.photos/200/300'),  
          Text('Hello World!'),  
        ],  
      ),  
    );  
  }  
}
```

MyWidget2(showImage: true)



MyWidget2(showImage: false)



MyWidget2.build(context)

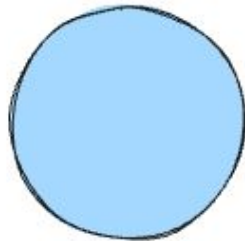


Should Flutter create new Elements?

Flutter compares:

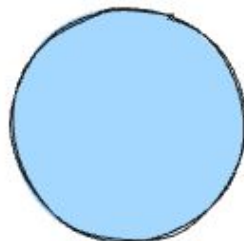
position in widget tree
identical (same object)
runtimeType
key

Widget A

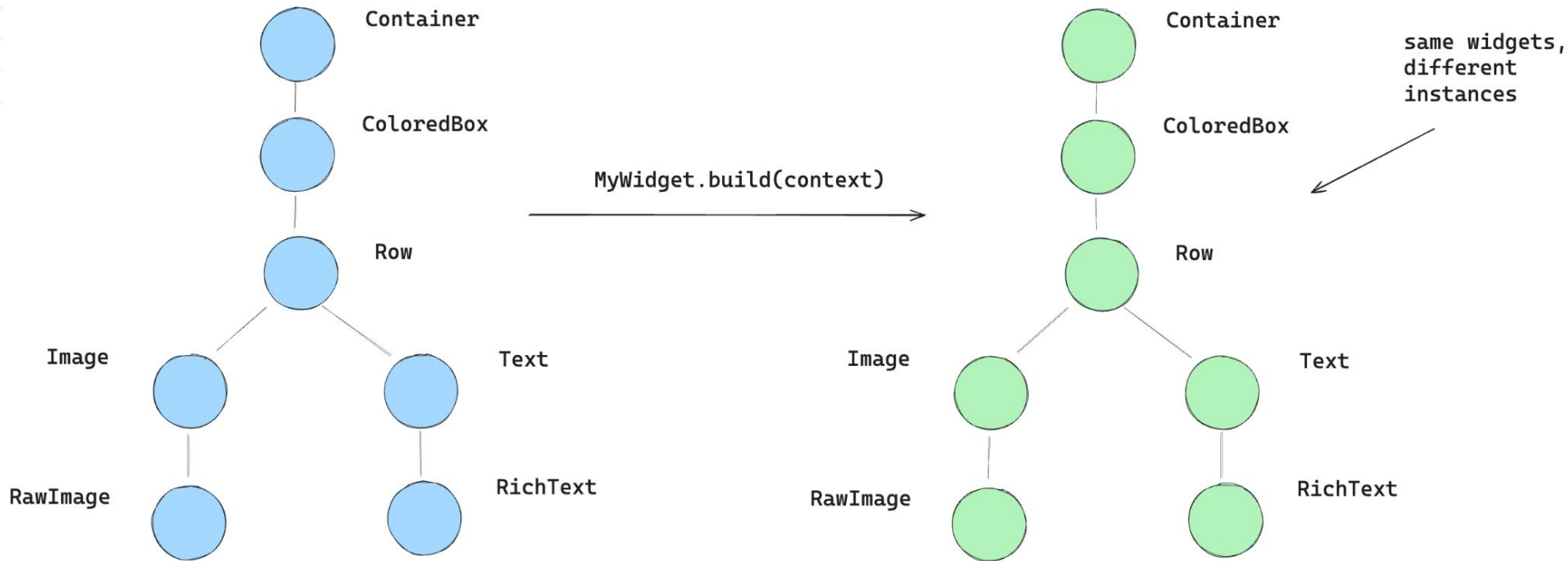


?
≡

Widget B

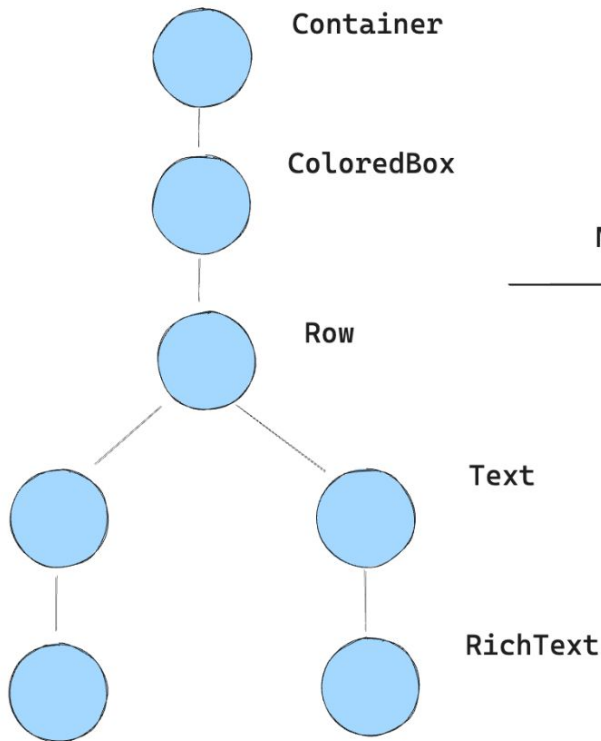


No changes to elements

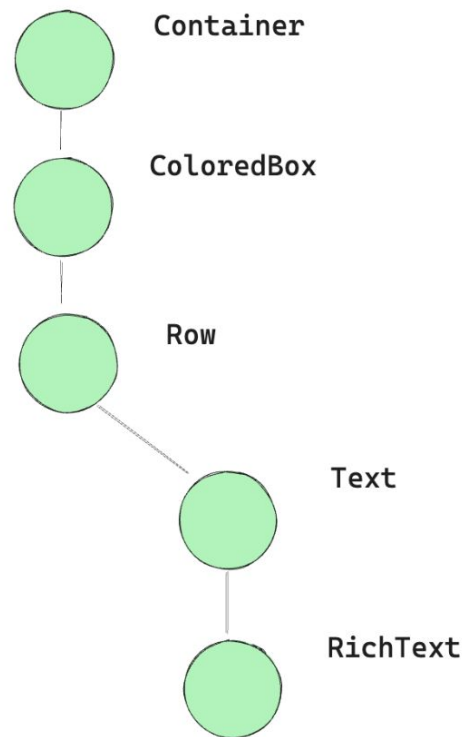


No changes to elements

MyWidget2(showImage: true)



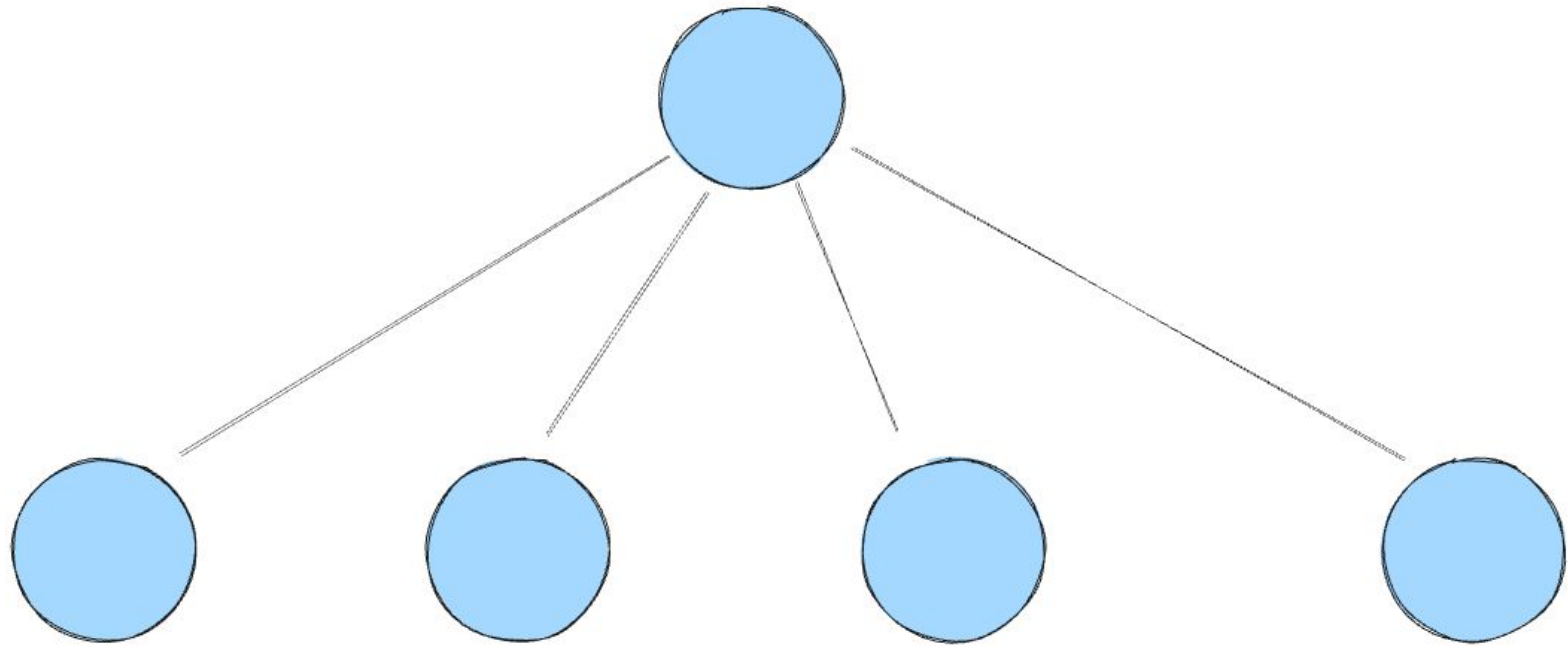
MyWidget2(showImage: false)



MyWidget2.build(context)



Registration Form



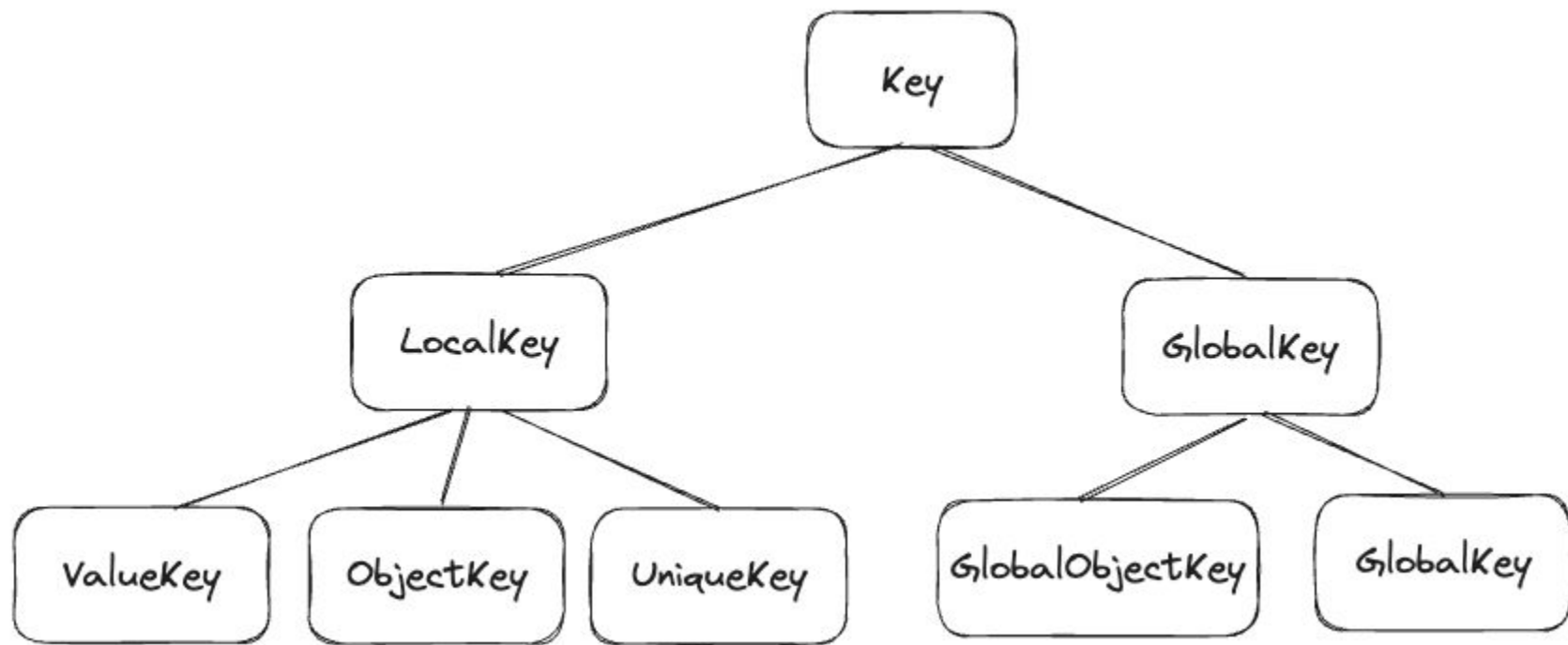
TextField A
(email)

TextField B
(password)

TextField C
(phone #)

TextField D
(billing address)

Key types



Key types

key1

key2

in Dart:

identical - same object/instance (referential equality)
 == - overridable equality comparison

ValueKey(value1)

ValueKey(value2)

key1 == key2 $\Leftrightarrow$ value1 == value2

ObjectKey(value1)

ObjectKey(value2)

key1 == key2 $\Leftrightarrow$ identical(value1, value2)

UniqueKey()

UniqueKey()

key1 == key2 $\Leftrightarrow$ identical(key1, key2)

GlobalKey()

GlobalKey()

key1 == key2 $\Leftrightarrow$ identical(key1, key2)

GlobalObjectKey(value1) GlobalObjectKey(value2)

key1 == key2 $\Leftrightarrow$ identical(value1, value2)

Key types

```

// 1)
ValueKey(5) == ValueKey(5) // true
ValueKey(5) == ValueKey(6) // false

// 2)
final key = ValueKey(5);
key == key; // true
key == ValueKey(5); // true

// 3)
Key('abc') == ValueKey('abc') // true -- Key is aliased to ValueKey<String>

// 4)
final obj = Object();
ValueKey(obj) == ValueKey(obj) // true
ValueKey(Object()) == ValueKey(Object()) // false -- two different objects

```

Key types



```
// 1)
ObjectKey(5) == ObjectKey(5) // true
ObjectKey(5) == ObjectKey(6) // false

// 2)
final key = ObjectKey(5);
key == key; // true
key == ObjectKey(5); // true

// 3)
Key('abc') == ObjectKey('abc') // false

// 4)
final obj = Object();
ObjectKey(obj) == ObjectKey(obj) // true
ObjectKey(Object()) == ObjectKey(Object()) // false -- two different objects
```

Key types



```
// 1)
UniqueKey() == UniqueKey() // false
```

```
// 2)
final key = UniqueKey();
key == key; // true
key == UniqueKey(); // false
```

Global keys

Use cases:

- transition widget between screens, maintaining state
 - [Hero transition](#)
- access state from outside via `GlobalKey.currentState`
- access element from outside via `GlobalKey.currentContext`

Good practices

Prefer stateless to stateful

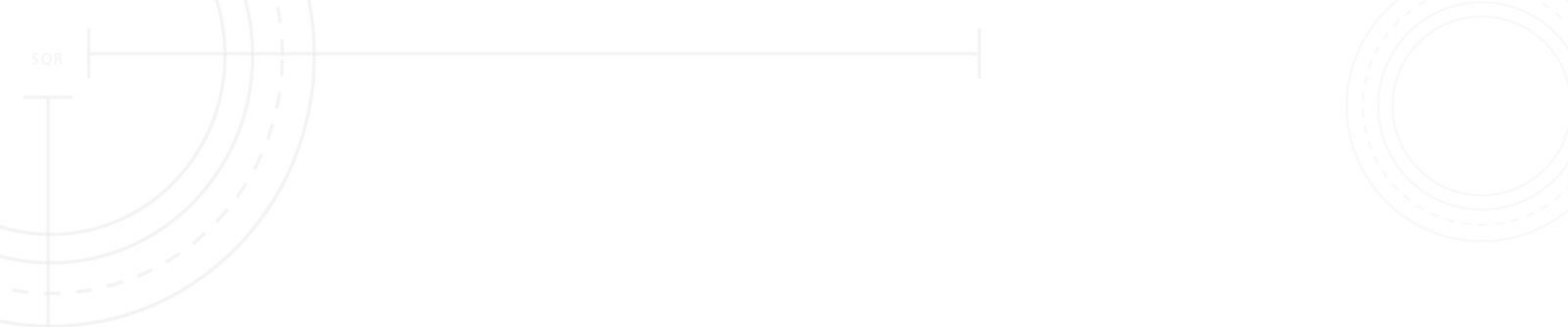
StatelessWidget is:

- cheaper in terms of performance (negligible; might make a difference at scale)
- more concise
- less effort to write
- vscode has quick actions to convert stateless $\longleftrightarrow$ stateful anyway

Don't overuse Provider/InheritedWidget

Passing values implicitly via InheritedWidget or Provider might be confusing to a code reviewer/maintainer/reader.

InheritedWidget forfeits static analysis of passing props directly and might result in runtime errors.

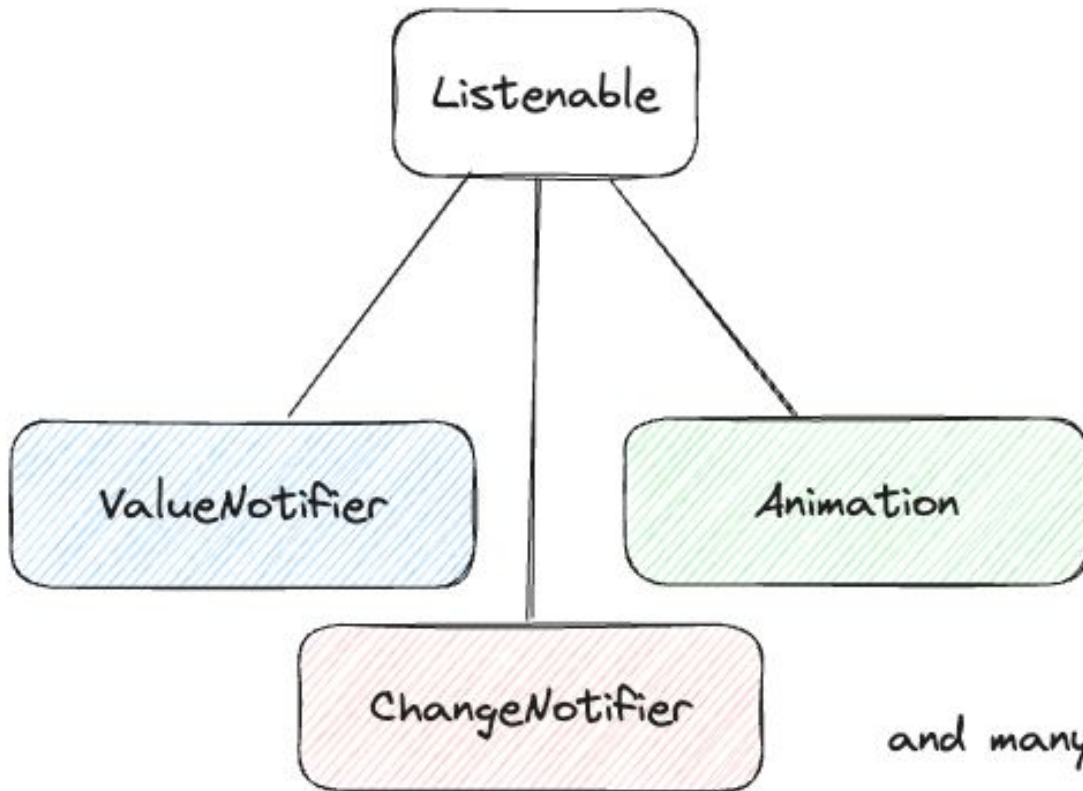


A note on observing state

Listenable

```
/// An object that maintains a list of listeners.  
/// [...]  
abstract class Listenable {  
    // [...]  
  
    /// Register a closure to be called when the object notifies its listeners.  
    void addListener(VoidCallback listener);  
  
    /// Remove a previously registered closure from the list of closures that the  
    /// object notifies.  
    void removeListener(VoidCallback listener);  
}
```

Types of listenables



and many more

ChangeNotifier

```
class LightSwitch extends ChangeNotifier {  
  bool _isOn = false;  
  bool get isOn => _isOn;  
  
  void switchOn() {  
    _isOn = true;  
    notifyListeners();  
  }  
  
  void switchOff() {  
    _isOn = false;  
    notifyListeners();  
  }  
}
```

ValueNotifier



```
void main(List<String> args) {  
  final notifier = ValueNotifier('foobar');  
  
  print(notifier.value);  
  
  notifier.value = 'abcdef';  
}
```

Subscribing to a Listenable

```
class MyWidget extends StatefulWidget {  
  const MyWidget({super.key, required this.notifier});  
  
  final ValueNotifier<String> notifier;  
  
  @override  
  State<MyWidget> createState() => _MyWidgetState();  
}  
  
class _MyWidgetState extends State<MyWidget> {  
  String _text = '';  
  
  void updateText() {  
    setState(() {  
      _text = widget.notifier.value;  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(_text);  
  }  
  
  // to be continued  
}
```

Subscribing to a Listenable

```
class _MyWidgetState extends State<MyWidget> {  
  // part 1 slide before  
  
  @override  
  void initState() {  
    super.initState();  
  
    _text = widget.notifier.value;  
    widget.notifier.addListener(updateText);  
  }  
  
  @override  
  void didUpdateWidget(MyWidget oldWidget) {  
    super.didUpdateWidget(oldWidget);  
  
    if (oldWidget.notifier != widget.notifier) {  
      oldWidget.notifier.removeListener(updateText);  
      widget.notifier.addListener(updateText);  
    }  
  }  
  
  @override  
  void dispose() {  
    widget.notifier.removeListener(updateText);  
    super.dispose();  
  }  
}
```

Common listenables

A lot of builtin widgets use a listenable as an additional “controller” object when using the widget tree is uncomfortable or not performant enough.

Examples:

- AnimationController – animations need that extra bit of performance
- ScrollController – all ScrollViews have one
- TextEditingController – handles more complex workflows involving text inputs

Polecajki do czytania

- [Flutter architectural overview](#)
 - [On state and inherited widgets](#)
 - [On elements and render objects](#)
- [On widget reconciliation algorithm](#)
- [Key class - foundation library - Dart API \(flutter.dev\)](#)
- [flutter hooks | Flutter Package](#) – alternative solution for managing state built on top of StatefulWidget
- [boxy | Flutter package](#) – easier way to write custom layouts (no need to create custom RenderObjects)